

test-angular-project

Audit date - 2026-05-06 00:28

Angular ^14.3.0 - 5 TS files - 204 LOC

0/100

Grade F

Critique - le projet necessite une intervention majeure.

48 issues detected

8

CRITICAL

31

IMPORTANT

9

MINOR

Project overview

Angular version	^14.3.0
TypeScript files	5
HTML templates	1
Components / Services / Modules	3 / 1 / 1
Pipes / Guards	0 / 0
Total lines of code	204
Tests detected	No

Outdated Angular version: ^14.3.0 (< 16)
No Signals, no Standalone, no Control Flow. Migrating to Angular 17+ is strongly recommended.

Severity summary



Issues detail

Memory Leaks

CRITICAL

MEM001 Subscription sans unsubscribe (4)

Une subscription sans unsubscribe/takeUntil crée un memory leak.

Fix: Utiliser `takeUntil(this.destroy$)` ou `takeUntilDestroyed()` (Angular 16+) ou le `async` pipe dans le template.

OCCURRENCES

```
src/app/components/user-list/user-list.component.ts:43
  this.http.get<User[]>('https://api.example.com/users').subscribe(
src/app/components/user-list/user-list.component.ts:58
  this.http.get('https://api.example.com/config').subscribe((config) => {
src/app/components/user-list/user-list.component.ts:72
  this.http.delete('https://api.example.com/users/${userId}').subscribe(() => {
src/app/components/user-list/user-list.component.ts:80
  this.http.get('https://api.example.com/users').subscribe();
```

IMPORTANT

JS001 setTimeout/setInterval sans cleanup (2)

Un `setTimeout` ou surtout `setInterval` lance dans un composant qui n'est jamais clear continue a tourner apres la destruction du composant. Sur une SPA Angular, accumuler des intervalles oublies = memory leak progressif + appels reseau fantomes. Different de RxJS subscriptions (gere par MEM001).

Fix: Garder la reference (`this.timerId = setTimeout(...)`) et appeler `clearTimeout(this.timerId)` dans `ngOnDestroy()`. Ou mieux : utiliser `interval(N).pipe(takeUntilDestroyed())` (Angular 16+) qui s'auto-nettoie.

OCCURRENCES

```
src/app/components/user-list/user-list.component.ts:33
  setInterval(() => {
src/app/components/user-list/user-list.component.ts:38
  setTimeout(() => {
```

Securite

CRITICAL**SEC001 innerHTML sans sanitization (2)**

innerHTML peut injecter du HTML malicieux (XSS). Angular bypass la sanitization avec innerHTML.

Fix: Utiliser ``DomSanitizer.bypassSecurityTrustHtml()`` avec validation stricte, ou restructurer le template sans innerHTML.

OCCURRENCES

[src/app/components/user-list/user-list.component.html:10](#)

```
<div class="error" [innerHTML]="errorMessage"></div>
```

[src/app/components/user-list/user-list.component.html:26](#)

```
<span [innerHTML]="user.name"></span>
```

CRITICAL**SEC002 Cle API ou secret hardcode (2)**

Une cle API, token ou secret hardcode dans le code source est expose des qu'il est commit. Toute personne ayant acces au repo (ou au bundle prod) peut l'extraire et abuser des credentials. Cas reel : OpenAI revoke automatiquement les sk-... detectees sur GitHub public.

Fix: Stocker dans ``src/environments/environment.ts`` (ignore par `.gitignore`) ou via variables d'env injectees au build (``ng build --configuration=production`` + ``fileReplacements``). Pour les secrets serveur, ne jamais les inclure cote client : passer par un backend proxy. Si la cle a deja ete commit, il faut la revoke immediatement (rotate) puis purger l'historique git.

OCCURRENCES

[src/app/components/user-list/user-list.component.ts:11](#)

```
const OPENAI_API_KEY = "sk-proj-abc123def456ghi789jkl012mno345pqr678stu901";
```

[src/app/components/user-list/user-list.component.ts:12](#)

```
const config = { apiKey: "sk-1234567890abcdefghijklmnopqrstuvwxyz1234567890abcdefghijklmnopqrstuvwxyz" };
```

Performance

IMPORTANT**PERF001 ChangeDetectionStrategy.Default (1)**

Default change detection verifie tous les composants à chaque cycle. Coûteux sur les grands arbres.

Fix: Utiliser ``ChangeDetectionStrategy.OnPush`` - fonctionne avec les Observables + async pipe et les Signals.

OCCURRENCES

[src/app/components/user-list/user-list.component.ts:18](#)

```
changeDetection: ChangeDetectionStrategy.Default, // PERF001: devrait être OnPush
```

IMPORTANT**PERF003 *ngFor sans trackBy (1)**

**ngFor sans trackBy force Angular à recréer tout le DOM à chaque détection de changement. Sur une liste de 100+ items qui change fréquemment, c'est un gros frein perf.*

Fix: Ajouter ``; trackBy: trackByFn`` dans le `*ngFor`, et définir ``trackByFn(index, item) { return item.id; }`` dans le composant. Sur Angular 17+, utiliser le new control flow ``@for`` avec ``track item.id``.

OCCURRENCES

[src/app/components/user-list/user-list.component.html:22](#)

```
<li *ngFor="let user of filteredUsers">
```

IMPORTANT**PERF002 Route sans lazy loading (3)**

Les routes chargées eagerly augmentent le bundle initial et ralentissent le démarrage.

Fix: Remplacer ``component: MyComponent`` par ``loadComponent: () => import('./my.component').then(m => m.MyComponent)``

OCCURRENCES

[src/app/app.module.ts:12](#)

```
{ path: '', component: DashboardComponent },
```

[src/app/app.module.ts:13](#)

```
{ path: 'users', component: UserListComponent },
```

[src/app/app.module.ts:14](#)

```
{ path: 'settings', component: UserListComponent }, // Réutilisation lazy
```

Type Safety

IMPORTANT

TYPE001 Usage de 'any' TypeScript (8)

Le type ``any`` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

Fix: Typer explicitement (interface, type, générique). Utiliser ``unknown`` si le type est vraiment inconnu.

OCCURRENCES

[src/app/services/user.service.ts:12](#)

```
getUsers(): Observable<any[]> { // TYPE001: any[] au lieu d'une interface User
```

[src/app/services/user.service.ts:16](#)

```
deleteUser(id: number): Observable<any> { // TYPE001: any
```

[src/app/components/user-list/user-list.component.ts:21](#)

```
users: any[] = []; // TYPE001: any au lieu de User[]
```

[src/app/components/user-list/user-list.component.ts:22](#)

```
filteredUsers: any = null; // TYPE001: any encore
```

[src/app/components/user-list/user-list.component.ts:24](#)

```
errorMessage: any; // TYPE001: any
```

[src/app/components/user-list/user-list.component.ts:65](#)

```
this.filteredUsers = this.users.filter((u: any) =>
```

[src/app/components/user-list/user-list.component.ts:70](#)

```
deleteUser(userId: any): void { // TYPE001: any
```

[src/app/components/user-list/user-list.component.ts:73](#)

```
this.users = this.users.filter((u: any) => u.id !== userId);
```

Architecture

IMPORTANT

ARCH001 HttpClient dans un composant (7)

Les appels HTTP dans les composants mélangent les responsabilités. Difficile à tester et réutiliser.

Fix: Déplacer les appels HTTP dans des services dédiés. Les composants ne consomment que des observables.

OCCURRENCES

[src/app/app.module.ts:4](#)

```
import { HttpClientModule } from '@angular/common/http';
```

[src/app/app.module.ts:25](#)

```
HttpClientModule,
```

[src/app/components/user-list/user-list.component.ts:2](#)

```
import { HttpClient } from '@angular/common/http';
```

[src/app/components/user-list/user-list.component.ts:26](#)

```
constructor(private http: HttpClient) {} // ARCH001: HttpClient directement dans le composant
```

[src/app/components/user-list/user-list.component.ts:58](#)

```
this.http.get('https://api.example.com/config').subscribe((config) => {
```

[src/app/components/user-list/user-list.component.ts:72](#)

```
this.http.delete(`https://api.example.com/users/${userId}`).subscribe(() => {
```

[src/app/components/user-list/user-list.component.ts:80](#)

```
this.http.get('https://api.example.com/users').subscribe();
```

IMPORTANT**ARCH002 URL hardcodée dans le code (5)**

Une URL hardcodée empêche de switcher entre dev/staging/prod sans rebuild. Force un commit pour changer un endpoint. Mauvaise pratique multi-environnements.

Fix: Déplacer l'URL dans `src/environments/environment.ts` et `environment.prod.ts`. Utiliser `environment.apiUrl` dans le code.

OCCURRENCES

[src/app/services/user.service.ts:8](#)

```
private apiUrl = 'https://api.example.com';
```

[src/app/components/user-list/user-list.component.ts:43](#)

```
this.http.get<User[]>('https://api.example.com/users').subscribe(
```

[src/app/components/user-list/user-list.component.ts:58](#)

```
this.http.get('https://api.example.com/config').subscribe((config) => {
```

[src/app/components/user-list/user-list.component.ts:72](#)

```
this.http.delete(`https://api.example.com/users/${userId}`).subscribe(() => {
```

[src/app/components/user-list/user-list.component.ts:80](#)

```
this.http.get('https://api.example.com/users').subscribe();
```

Accessibilité

IMPORTANT**A11Y001 Image sans attribut alt (2)**

Une balise `<img>` sans attribut `alt` est invisible aux lecteurs d'écran et pénalise le SEO. Erreur a11y la plus commune.

Fix: Ajouter ``alt="description courte"`` (ou ``alt=""`` pour les images purement décoratives). Pour les images dynamiques, binder ``[alt]="item.label"``.

OCCURRENCES

[src/app/components/user-list/user-list.component.html:7](#)

```

```

[src/app/components/user-list/user-list.component.html:24](#)

```
<img [src]="user.avatar">
```

IMPORTANT**A11Y002 Click sur element non-interactif (2)**

Un (click) sur un `<div>` ou `<span>` sans role ni tabindex est inaccessible au clavier et aux lecteurs d'écran. L'utilisateur ne peut pas activer l'action sans souris.

Fix: Soit utiliser un vrai `<button>` (ou `<a>` si c'est une navigation), soit ajouter ``role="button" tabindex="0"`` + handler ``(keydown.enter)`` et ``(keydown.space)``.

OCCURRENCES

[src/app/components/user-list/user-list.component.html:19](#)

```
<div class="custom-button" (click)="refresh()">Actualiser</div>
```

[src/app/components/user-list/user-list.component.html:29](#)

```
<span class="action-link" (click)="viewProfile(user.id)">voir profil</span>
```

Code Quality

DEBUG001 console.log en production (6)

Les console.log oubliés exposent des données internes en prod et polluent la console.

Fix: Supprimer ou remplacer par un service de logging. Configurer `build.optimization.scripts` pour stripper en prod.

OCCURRENCES

```
src/app/components/user-list/user-list.component.ts:30
  console.log('UserListComponent initialized'); // DEBUG001: console.log oublié

src/app/components/user-list/user-list.component.ts:39
  console.log('lazy init done');
```

```
src/app/components/user-list/user-list.component.ts:48
  console.log('Users loaded:', data.length); // DEBUG001: encore un console.log

src/app/components/user-list/user-list.component.ts:51
  console.error('Error loading users:', error);

src/app/components/user-list/user-list.component.ts:59
  console.log('Config loaded', config); // DEBUG001

src/app/components/user-list/user-list.component.ts:74
  console.log('User deleted:', userId); // DEBUG001
```

MINOR

TEST001 Test skippe ou focus (3)

Un `xit`, `fit`, `it.skip` ou `describe.only` oublié signifie qu'une partie de la suite de tests est désactivée ou que la CI ne passe que sur un sous-ensemble. Risque de régression silencieuse.

Fix: Soit corriger et réactiver le test (`xit` ? `it`), soit supprimer la suite si elle n'est plus pertinente. Le focus (`fit`, `describe.only`) doit toujours être retiré avant commit.

OCCURRENCES

```
src/app/components/user-list/user-list.component.spec.ts:18
  xit('should filter users by search term', () => {

src/app/components/user-list/user-list.component.spec.ts:23
  fit('should delete a user', () => {

src/app/components/user-list/user-list.component.spec.ts:28
  it.skip('should refresh the list', () => {
```

Lazy loading

Eager routes (no lazy)	3
Lazy routes	0
Lazy loading ratio	0%

Less than 50% of routes use lazy loading. Each eager route adds to the initial bundle. Migrate to loadComponent (Angular 15+) for the heaviest routes first.

Refactoring plan

This week - Critical

- Subscription sans unsubscribe (MEM001)
Une subscription sans unsubscribe/takeUntil crée un memory leak.
- innerHTML sans sanitization (SEC001)
innerHTML peut injecter du HTML malicieux (XSS). Angular bypass la sanitization avec innerHTML.
- Cle API ou secret hardcoded (SEC002)

Une cle API, token ou secret hardcoded dans le code source est expose des qu'il est commit. Toute personne ayant acces au repo (...)

This month - Important

- ChangeDetectionStrategy.Default (PERF001)

Default change detection vérifie tous les composants à chaque cycle. Coûteux sur les grands arbres.

- Usage de 'any' TypeScript (TYPE001)

Le type `any` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

- HttpClient dans un composant (ARCH001)

Les appels HTTP dans les composants mélangent les responsabilités. Difficile à tester et réutiliser.

- *ngFor sans trackBy (PERF003)

*ngFor sans trackBy force Angular à recréer tout le DOM à chaque détection de changement. Sur une liste de 100+ items qui chang...

- URL hardcodée dans le code (ARCH002)

Une URL hardcodée empêche de switcher entre dev/staging/prod sans rebuild. Force un commit pour changer un endpoint. Mauvaise...

- Image sans attribut alt (A11Y001)

Une balise sans attribut alt est invisible aux lecteurs d'écran et pénalise le SEO. Erreur a11y la plus commune.

- Click sur element non-interactif (A11Y002)

Un (click) sur un <div> ou sans role ni tabIndex est inaccessible au clavier et aux lecteurs d'écran. L'utilisateur ne p...

- setTimeout/setInterval sans cleanup (JS001)

Un `setTimeout` ou surtout `setInterval` lance dans un composant qui n'est jamais clear continue a tourner apres la destruction...

- Route sans lazy loading (PERF002)

Les routes chargées eagerly augmentent le bundle initial et ralentissent le démarrage.

On the roadmap - Minor

- console.log en production (DEBUG001)

Les console.log oubliés exposent des données internes en prod et polluent la console.

- Test skippe ou focus (TEST001)

Un `xit`, `fit`, `it.skip` ou `describe.only` oublié signifie qu'une partie de la suite de tests est désactivée ou que la CI ne...

This report is produced by automated static analysis. It does not replace a thorough manual review by an experienced Angular developer.